



Instituto Tecnológico
de Buenos Aires

Infraestructura como código (IaC)

82.08 Cloud Computing

—

DEEP DIVE - Sintaxis HCL

HCL (HashiCorp Configuration Language)



- **Bloques:** Contenedores para configuraciones.
 - `terraform {}`, `provider {}`, `resource {}`, `variable {}`, `output {}`, `data {}`, `module {}`
 - Sintaxis: `tipo_bloque "etiqueta_1" "etiqueta_2" { ... argumentos ... }`
- **Argumentos:** Asignan valores dentro de un bloque.
 - Sintaxis: `nombre_argumento = valor`
- **Expresiones:** Representan valores, pueden ser literales o referencias.
 - Literales: `"string"`, `123`, `true`, `["a", "b"]`, `{key = "value"}`
 - Referencias: `var.nombre_variable`, `aws_instance.mi_instancia.id`

DEEP DIVE - Sintaxis HCL Ejemplo

```
# Bloque de configuración de Terraform  
terraform { ... }
```

```
# Bloque de proveedor  
provider "aws" { # "aws" es la etiqueta_1 (nombre del proveedor)  
  region = "us-east-1" # argumento y valor  
}
```

```
# Bloque de recurso  
resource "aws_s3_bucket" "mi_bucket_demo" { # "aws_s3_bucket" es tipo, "mi_bucket_demo" es etiqueta_1 (nombre local)  
  bucket = "mi-bucket-unico" # argumento y valor  
}
```

DEEP DIVE - Sintaxis HCL - Variables

Propósito:

- Parametrizar configuraciones para hacerlas reutilizables y flexibles.
- Evitar "hardcodear" valores.

```
variable "nombre_variable" {
  description = "Descripción de la variable."
  type        = string # 0 number, bool, list, map, set, object, tuple, any
  default     = "valor_por_defecto" # Opcional
  sensitive   = false # Opcional, true para ocultar en logs
  # También: validation {} para reglas personalizadas
}
```

Ejemplo declaración

```
variable "aws_region" {
  description = "Región de AWS para desplegar los recursos."
  type        = string
  default     = "us-east-1"
}

variable "instance_type" {
  description = "Tipo de instancia EC2."
  type        = string
}
```

Ejemplo uso

```
provider "aws" {
  region = var.aws_region
}

resource "aws_instance" "web" {
  ami           = "ami-xxxxxxxxxxxxxxxxxxxx" # Un AMI
  instance_type = var.instance_type
}
```

DEEP DIVE - Sintaxis HCL - Variables

Formas de asignar valores:

- Argumentos en línea de comando: `-var="nombre_variable=valor"`
- Archivos `.tfvars` o `.tfvars.json`: `terraform apply -var-file="testing.tfvars"`
- Archivos `terraform.tfvars` o `terraform.tfvars.json` (cargados automáticamente).
- Variables de entorno: `TF_VAR_nombre_variable=valor`
- Valor `default` en la declaración.

DEEP DIVE - Sintaxis HCL - Output

Propósito:

- Exponer información sobre la infraestructura gestionada.
- Pasar datos a otros espacios de trabajo de Terraform o herramientas externas.
- Mostrar información útil al usuario después de un apply.

```
output "nombre_salida" {
  description = "Descripción de la salida."
  value       = expresion # Ej: aws_instance.web.public_ip
  sensitive   = false # Opcional, true para ocultar en la salida estándar
  # depends_on = [resource.a, resource.b] # Opcional
}
```

Ejemplo declaración

```
resource "aws_instance" "web_server" {
  # ... configuración de la instancia ...
  ami           = "ami-0c55642e3f7a8d3d9" # Amazon Linux 2
  instance_type = "t2.micro"
}

output "web_server_public_ip" {
  description = "Dirección IP pública del servidor web."
  value       = aws_instance.web_server.public_ip
}

output "web_server_id" {
  description = "ID de la instancia del servidor web."
  value       = aws_instance.web_server.id
}
```

Acceso:

- Se muestran automáticamente después de un terraform apply.
- Comando: terraform output nombre_salida
- En otros workspaces de Terraform usando terraform_remote_state.

DEEP DIVE - Sintaxis HCL - Datasources

Propósito:

- Obtener información sobre recursos existentes que NO son gestionados por la configuración actual de Terraform.
- Permite que tu configuración de Terraform utilice datos definidos fuera de Terraform.

Uso común:

- Obtener IDs de VPCs, subreds, AMIs, zonas de disponibilidad, etc.
- Leer secretos de un gestor de secretos.

```
data "aws_ami" "amazon_linux_2" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }

  filter {
    name     = "virtualization-type"
    values   = ["hvm"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux_2.id
  instance_type = "t2.micro"
  # ...
}
```

DEEP DIVE - Sintaxis HCL - Funciones

Propósito:

- Transformar y combinar valores dentro de la configuración.
- Permiten lógica más compleja sin recurrir a scripts externos.

Uso común:

Numéricas: `min()`, `max()`, `parseint()`

Cadenas (Strings): `format()`, `join()`, `split()`, `lower()`, `upper()`, `substr()`

Colecciones (Lists/Maps/Sets): `length()`, `lookup()`, `merge()`, `concat()`, `element()`, `keys()`, `values()`

Codificación: `jsonencode()`, `jsondecode()`, `base64encode()`

Sistema de Archivos: `file()`, `pathexpand()`

Fecha y Hora: `timestamp()`, `formatdate()`

Redes IP: `cidrsubnet()`, `cidrhost()`

Conversión de Tipos: `tostring()`, `tonumber()`, `tobool()`

```
resource "aws_instance" "web" {  
  # ...  
  tags = merge(  
    local.common_tags,  
    {  
      Name = local.instance_name  
    }  
  )  
  # Ejemplo de uso de file() para user_data  
  # user_data = file("${path.module}/user_data.sh")  
}
```


DEEP DIVE - Sintaxis HCL - Meta Argumentos

Propósito:

- Argumentos especiales que cambian el comportamiento de los recursos, no son específicos del recurso en sí.
- Se pueden usar en cualquier bloque `resource` (y algunos en `module` o `data`).

count:

- Crea múltiples instancias de un recurso basadas en un número.
- Útil para crear N servidores idénticos.
- Acceso a instancias individuales:
`aws_instance.example[0]`,
`aws_instance.example[1]`

```
variable "num_web_servers" { type = number; default = 2 }
resource "aws_instance" "web" {
  count = var.num_web_servers
  ami    = "ami-..."
  instance_type = "t2.micro"
  tags = { Name = "WebServer-${count.index + 1}" }
}
```

for_each:

- Crea múltiples instancias basadas en un `map` o `set` de strings.
- Más flexible que `count` para recursos con configuraciones ligeramente diferentes.
- Acceso: `aws_instance.example["key1"]`,
`aws_s3_bucket.example_set["valueA"]`

```
variable "s3_bucket_names" {
  type    = map(object({ enable_versioning = bool }))
  default = {
    "bucket-alpha" = { enable_versioning = true },
    "bucket-beta"  = { enable_versioning = false }
  }
}

resource "aws_s3_bucket" "data_store" {
  for_each = var.s3_bucket_names
  bucket   = each.key
  # Acceder a atributos del objeto del map
  # versioning { enabled = each.value.enable_versioning }
}
```

DEEP DIVE - Sintaxis HCL - Meta Argumentos

depends_on:

- Define dependencias explícitas que Terraform no puede inferir automáticamente.
- Ej: `depends_on = [aws_iam_role_policy_attachment.example]`

provider:

- Especifica qué configuración de proveedor usar si hay múltiples alias para el mismo proveedor (ej. diferentes regiones AWS).
- Ej: `provider = aws.secondary_region`

lifecycle block: controla el ciclo de vida del recurso

- `create_before_destroy`: Crea el nuevo recurso antes de destruir el viejo (útil para evitar downtime).
- `prevent_destroy`: Evita que el recurso sea destruido accidentalmente.
- `ignore_changes`: Ignora cambios en atributos específicos fuera de Terraform.

DEEP DIVE - Sintaxis HCL - Módulos

Propósito:

- Encapsular y reutilizar configuraciones de Terraform.
- Organizar código complejo en unidades lógicas.
- Abstraer detalles de implementación.

¿Qué es?:

- Cualquier conjunto de archivos `.tf` en un directorio es un módulo.
- El directorio raíz de tu configuración es el "módulo raíz".

Estructura Típica de un Módulo (Recomendada):

- `main.tf`: Recursos principales.
- `variables.tf`: Definiciones de variables de entrada.
- `outputs.tf`: Definiciones de salidas del módulo.
- `versions.tf`: Requisitos de versión de Terraform y proveedores.
- `README.md`: Documentación.

```
module "nombre_modulo_local" {  
    source = "../ruta/al/modulo" # Módulo local  
  
    # Variables de entrada para el módulo  
    variable1 = "valor"  
    variable2 = var.alguna_otra_variable  
}  
  
module "vpc" {  
    source = "terraform-aws-modules/vpc/aws" # Módulo del Registry  
    version = "5.5.1" # Siempre especificar versión  
  
    name = "mi-vpc-de-modulo"  
    cidr = "10.0.0.0/16"  
    # ... otras variables del módulo VPC  
}
```

Acceso a Salidas de Módulos:

`module.nombre_modulo_local.nombre_salida_del_modulo`

DEEP DIVE - Sintaxis HCL - Módulos

Beneficios

- DRY (Don't Repeat Yourself).
- Mejora la mantenibilidad y la colaboración.
- Permite la creación de "blueprints" de infraestructura.

Fuentes de módulos (source)

- **Locales:** Rutas en el sistema de archivos (`./modules/my_module`).
- **Terraform Registry (Público y Privado):**
`terraform-aws-modules/vpc/aws`.
- **Git:** `git::https://example.com/vpc.git?ref=v1.2.0`.
- Otros (HTTP, S3, GCS).



Instituto Tecnológico
de Buenos Aires

¡MUCHAS
GRACIAS!

MÁS INFORMACIÓN > www.itba.edu.ar